PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
INSTITUTO DE CIÊNCIAS EXATAS E INFORMÁTICA
ENGENHARIA DE SOFTWARE

Disciplina: Teste de Software

ANÁLISE DE EFICÁCIA DE TESTES COM TESTE DE MUTAÇÃO

Avaliação e melhoria de uma suíte Jest com StrykerJS

Leonardo de Freitas Viana
Matrícula: 802807
Repositório: github.com/VianaLeo13/operacoes-mutante

Belo Horizonte
2026

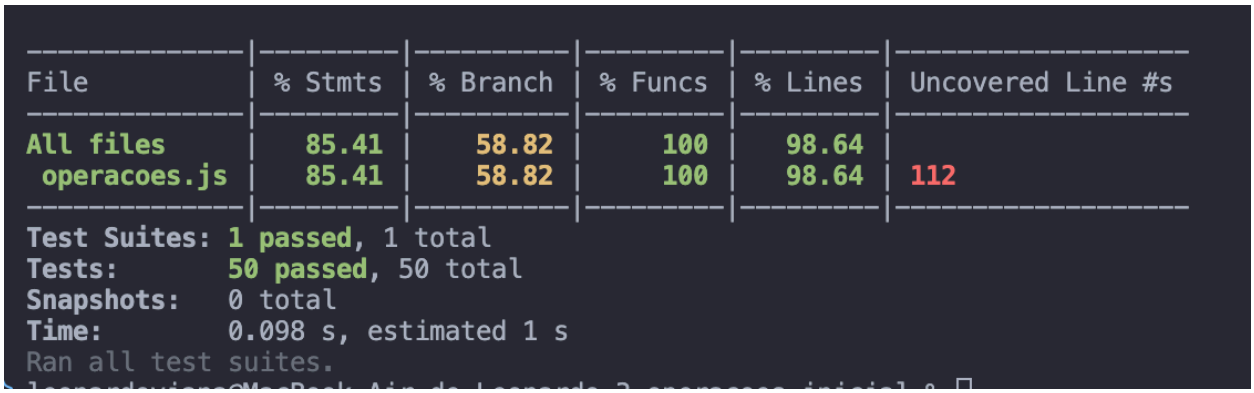
1 ANÁLISE INICIAL

O PROJETO TEM 50 FUNÇÕES EM `src/operacoes.js` E UMA SUÍTE JEST COM 50 TESTES, UM POR FUNÇÃO, TODOS PASSANDO. ANTES DE ALTERAR QUALQUER COISA, MEDI A COBERTURA COM `npm test -- --coverage` E RODEI O STRYKERJS (V10, `jest-runner`, `coverageAnalysis: perTest`).

TABELA 1 – SITUAÇÃO DA SUÍTE ORIGINAL

Métrica	Valor inicial
Cobertura de linhas / funções	98,64% / 100%
Cobertura de statements / branches	85,41% / 58,82%
Mutantes mortos / timeout / sobreviventes / sem cobertura	154 / 3 / 44 / 12 (213)
Mutation score	73,71%

FONTE: ELABORADO PELO AUTOR.



QUASE TODAS AS LINHAS RODAM, MAS CERCA DE UM EM CADA QUATRO DEFEITOS INTRODUZIDOS PELO STRYKER PASSA DESPERCEBIDO. A COBERTURA SÓ DIZ SE O TRECHO FOI EXECUTADO, NÃO SE O RESULTADO FOI CONFERIDO DE UM JEITO CAPAZ DE SEPARAR O COMPORTAMENTO CERTO DO ERRADO. NA SUÍTE ORIGINAL ISSO APARECIA DE TRÊS FORMAS: PREDICADOS TESTADOS SÓ COM ENTRADAS QUE RETORNAM `true`; NENHUM VALOR DE FRONTEIRA (`a === b`, `n = 0`, `n = 1`); E `toThrow()` SEM CONFERIR A MENSAGEM, COM ALGUNS RAMOS DE ERRO NEM SENDO EXECUTADOS.

2 ANÁLISE DE MUTANTES CRÍTICOS

2.1 isMaiorQue (EqualityOperator e ConditionalExpression)

```
- return a > b;
+ return a >= b;   // ou: return true;
```

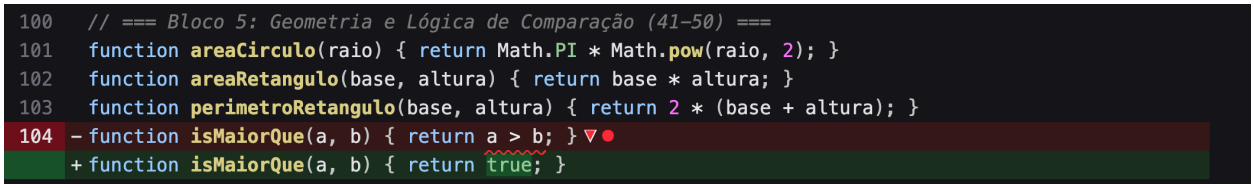


Figura 1 – Mutante sobrevivente em isMaiorQue (linha 104)

O único teste era `isMaiorQue(10, 5)` esperando `true`. Para essa entrada, `a > b`, `a >= b` e `true` dão o mesmo resultado, então a troca passa sem ser notada. Faltava um caso com `a === b`, onde > e >= divergem, e um caso com resposta `false`. O mesmo acontecia em `isMenorQue`, `isEqual`, `isPar`, `isImpar` e `isDivisivel`.

2.2 isPrimo (laço e condição de parada)

```
if (n <= 1) return false;          // n < 1 | if (false)
for (let i = 2; i < n; i++) {      // i >= n | corpo removido
  if (n % i === 0) return false;   // if (false) | n * i | return true
}
```

```
71  }
72  function isPrimo(n) {
73    if (n <= 1) return false;
74  -   for (let i = 2; i < n; i++) {
75  -     if (n % i === 0) return false;
76  -   }
    +   for (let i = 2; i < n; i++) {}
77    return true;
78  }
79  function fibonacci(n) { // Retorna o n-ésimo termo
80    if (n <= 1) return n;
81    return fibonacci(n - 1) + fibonacci(n - 2);
```

Figura 2 – Mutantes sobreviventes em isPrimo (linhas 73 a 75)

O teste só verificava `isPrimo(7) === true`. Como 7 é primo, o laço nunca encontra divisor, então remover o corpo, inverter a condição ou mexer no `return false` interno não muda a resposta. A guarda `n <= 1` também nunca era testada com 0, 1 ou negativos. A suíte aceitaria uma função que diz que todo número é primo: 9 mutantes sobreviveram ou ficaram sem cobertura só aqui.

2.3 medianaArray (MethodExpression e ArrowFunction)

```
- const sorted = [...numeros].sort((a, b) => a - b);
+ const sorted = [...numeros];   // ou: .sort((a, b) => a + b)
```

```
107 function medianaArray(numeros) {
108   if (numeros.length === 0) throw new Error('Array vazio не possui mediana.');
109 -   const sorted = [...numeros].sort((a, b) => a - b);
    +   const sorted = [...numeros];
110   const mid = Math.floor(sorted.length / 2);
111   if (sorted.length % 2 === 0) {
112     return (sorted[mid - 1] + sorted[mid]) / 2;
113   }
114   return sorted[mid];
115 }
```

Figura 3 – Mutante sobrevivente em medianaArray (linha 109)

O teste usava `[1, 2, 3, 4, 5]`, que já está ordenado, então tirar a ordenação não muda nada. Por ter tamanho ímpar, o ramo de tamanho par ficou sem cobertura, e a mensagem de erro para array vazio nunca foi conferida.

3 SOLUÇÃO IMPLEMENTADA

ADICIONEI 27 TESTES (51 A 77) EM UM NOVO `DESCRIBE`, SEM ALTERAR O CÓDIGO-FONTE NEM OS TESTES ORIGINAIS. OS QUE ATACAM OS MUTANTES ACIMA:

```
EXPECT(ISMAIORQUE(5, 5)).TOBE(FALSE);        // MATA A >= B

EXPECT(ISMAIORQUE(5, 10)).TOBE(FALSE);       // MATA RETURN TRUE

EXPECT(ISPRIMO(1)).TOBE(FALSE);              // MATA N < 1 (TAMBÉM 0 E -7)

EXPECT(ISPRIMO(9)).TOBE(FALSE);              // OBRIGA O LAÇO A ACHAR DIVISOR

EXPECT(ISPRIMO(2)).TOBE(TRUE);               // MENOR PRIMO

EXPECT(MEDIANAARRAY([3, 1, 2])).TOBE(2);     // ORDENAÇÃO VIRA OBSERVÁVEL

EXPECT(MEDIANAARRAY([5, 1, 4, 2])).TOBE(3); // RAMO PAR + DESORDENADO
```

CADA ENTRADA FOI ESCOLHIDA PARA QUE O CÓDIGO ORIGINAL E O MUTADO DEEM RESULTADOS DIFERENTES. `ISMAIORQUE(5, 5)` É O PONTO EXATO EM QUE > E >= SE SEPARAM. `ISPRIMO(9)` FORÇA O LAÇO A ENCONTRAR O DIVISOR 3, ENTÃO REMOVER O LAÇO, INVERTER A CONDIÇÃO OU TROCAR `N % I` POR `N * I` FAZ A FUNÇÃO RESPONDER `TRUE`. OS ARRAYS DESORDENADOS TORNAM A ORDENAÇÃO VISÍVEL, E O DE TAMANHO PAR AINDA MATA OS MUTANTES EM `MID - 1`, NO + E NO `/ 2`.

OS DEMAIS TESTES SEGUEM A MESMA LÓGICA: MENSAGENS EXATAS EM `TOTHROW('...')`, FRONTEIRAS COMO `FATORIAL(0)`, `RAIZQUADRADA(0)` E `MEDIAARRAY([])`, CASOS `FALSE` COMO `ISPAR(7)` E `ISEQUAL(7, 8)`, E CONVERSÕES COM 100 °C E 212 °F (COM 0 °C, TROCAR `9/5` POR `9*5` NÃO MUDA NADA). DOIS FORAM MENOS ÓBVIOS: `CLAMP(-0, 0, 10)`, QUE SEPARA < DE <= PORQUE O `TOBE` USA `OBJECT.IS`, E UM MOCK EM `PRODUTOARRAY` PARA PROVAR QUE O ATALHO DE ARRAY VAZIO EVITA A CHAMADA AO `REDUCE`.

4 RESULTADOS FINAIS

TABELA 2 – ANTES E DEPOIS

Métrica	Inicial	Final
Número de testes	50	77
Cobertura de statements / branches	85,41% / 58,82%	100% / 100%
Mutantes mortos (com timeout)	157	209
Sobreviventes / sem cobertura	44 / 12	4 / 0
Mutation score	73,71%	98,12%

FONTE: ELABORADO PELO AUTOR.

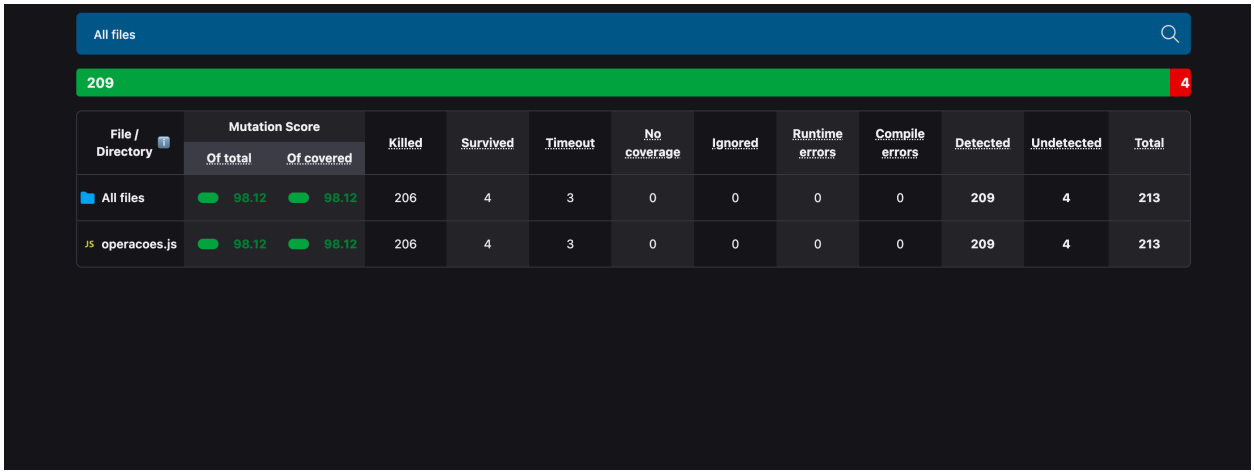


FIGURA 4 – RELATÓRIO FINAL DO STRYKER COM MUTATION SCORE DE 98,12%

O MUTATION SCORE SUBIU DE 73,71% PARA 98,12%. OS 4 SOBREVIVENTES ESTÃO TODOS EM `IF (N === 0 || N === 1) RETURN 1;` DE `FATORIAL` E SÃO EQUIVALENTES: SEM A GUARDA, O LAÇO `FOR (I = 2; I <= N; ...)` NÃO EXECUTA PARA 0 OU 1 E A FUNÇÃO DEVOLVE 1 DO MESMO JEITO. NENHUM TESTE CONSEGUE DISTINGUI-LOS, E ELIMINÁ-LOS EXIGIRIA MUDAR O CÓDIGO-FONTE, O QUE ESTAVA FORA DO ESCOPO.

5 CONCLUSÃO

O TRABALHO MOSTROU QUE COBERTURA E EFICÁCIA DE TESTES SÃO COISAS DIFERENTES. COM QUASE 99% DE LINHAS COBERTAS, A SUÍTE ORIGINAL ACEITARIA UMA FUNÇÃO EM QUE TODO NÚMERO É PRIMO OU EM QUE `ISMAIORQUE(5, 5)` RETORNA `TRUE`. A COBERTURA RESPONDE "ESSE CÓDIGO RODOU?", ENQUANTO O TESTE DE MUTAÇÃO PERGUNTA "SE ESSE CÓDIGO ESTIVESSE ERRADO, ALGUM TESTE IA PERCEBER?".

NA PRÁTICA, O RELATÓRIO DO STRYKER FUNCIONOU COMO ROTEIRO: CADA SOBREVIVENTE APONTAVA UM CASO QUE FALTAVA, QUASE SEMPRE UMA FRONTEIRA, UM RESULTADO `FALSE` OU UM RAMO DE ERRO. DE QUEBRA, REVELOU CÓDIGO REDUNDANTE (A GUARDA DE `FATORIAL`) E MENSAGENS DE ERRO COM A PALAVRA RUSSA "HE" NO LUGAR DE "NÃO", QUE AGORA FICAM FIXADAS NOS TESTES. O CUSTO É O TEMPO DE EXECUÇÃO E A ANÁLISE MANUAL DOS EQUIVALENTES, MAS PARA CÓDIGO CRÍTICO VALE A PENA.